
Lab 1 – Using Testbench and Waveform Debugging

Lab 1 – COMPENG 4DV4

*Chris Hoftyzer 400410904 - Group
12*

Checkpoint 1:	2
Checkpoint 2:	4
Checkpoint 3:	5

Checkpoint 1:

```
ncverilog Lab1_alu_tb.sv Lab1_alu.v
TOOL:  xmverilog    24.03-s013: Started on Jan 20, 2026 at 12:51:13 EST
TOOL:  xmverilog    24.03-s013: Started on Jan 20, 2026 at 12:51:13 EST
xmverilog(64): 24.03-s013: (c) Copyright 1995-2025 Cadence Design Systems, Inc.
xmverilog: *W,NCEXDEP: Executable (ncverilog) is deprecated. Use (xmverilog) instead.
file: Lab1_alu_tb.sv
  module worklib.tb_alu_verify:sv
    errors: 0, warnings: 0
    Caching library 'worklib' ..... Done
    Elaborating the design hierarchy:
    Building instance overlay tables: .....
    $fatal("ERROR: Could not open alu_test_vectors.hex");
    |
xmclab: *W,STRINT (./Lab1_alu_tb.sv,48|62): String literal argument supplied to integer parameter.
..... Done
  Generating native compiled code:
    worklib.alu:v <0x780e8796>
      streams: 2, words: 1145
    worklib.tb_alu_verify:sv <0x702263e1>
      streams: 6, words: 8723
  Building instance specific data structures.
  Loading native compiled code: ..... Done
  Design hierarchy summary:
      Instances  Unique
Modules:         2      2
Registers:       12     12
Scalar wires:    2      -
Vectored wires:  5      -
Always blocks:   1      1
Initial blocks:  1      1
Cont. assignments: 1      1
Pseudo assignments: 5      -
Simulation timescale: 1ps
  Writing initial simulation snapshot: worklib.tb_alu_verify:sv
Loading snapshot worklib.tb_alu_verify:sv ..... Done
xcelium> source /CMC/tools/cadence/XCELIUMMAIN24.03.lnx86/tools/xcelium/files/xmsimrc
xcelium> run
```

```
=== ALU VERIFICATION START ===
FAIL @ line 1
  op=000 a=0x0000000a b=0x00000005
  expected: result=0x0000000f zero=0
  got      : result=0x00000005 zero=0
FAIL @ line 2
  op=000 a=0xffffffff b=0x00000001
  expected: result=0x00000000 zero=1
  got      : result=0xffffffff zero=0
PASS @ line 3
PASS @ line 4
PASS @ line 5
PASS @ line 6
PASS @ line 7
PASS @ line 8
PASS @ line 9
PASS @ line 10
PASS @ line 11
PASS @ line 12
PASS @ line 13
PASS @ line 14
PASS @ line 15
PASS @ line 16
PASS @ line 17
PASS @ line 18
PASS @ line 19
PASS @ line 20
FAIL @ line 21
  op=000 a=0xe2ca4ec5 b=0x2e58495c
  expected: result=0x11229821 zero=0
  got      : result=0xb4720569 zero=0
PASS @ line 22
PASS @ line 23
FAIL @ line 24
  op=000 a=0x557845aa b=0xcceccc0d
  expected: result=0x24451247 zero=0
  got      : result=0x86ab790d zero=0
PASS @ line 25
PASS @ line 26
PASS @ line 27
PASS @ line 28
PASS @ line 29
PASS @ line 30
FAIL @ line 31
  op=000 a=0xc48a1289 b=0x75c50deb
  expected: result=0x3a4f2074 zero=0
  got      : result=0x4ec5049e zero=0
PASS @ line 32
=== FAILED: 5 errors ===
Simulation complete via $finish(1) at time 32 NS + 0
./Lab1_alu_tb.sv:95 $finish;
xcelium> exit
TOOL:  xmverilog    24.03-s013: Exiting on Jan 20, 2026 at 12:51:14 EST (total: 00:00:01)
[1] + Done gedit Lab1_alu.v
```

Figure 1 – Terminal output showing successful compilation

```
`timescale 1ns/1ps

module alu #(
    parameter WIDTH = 32
)(
    input [WIDTH-1:0] a,
    input [WIDTH-1:0] b,
    input [2:0] alu_op,
    output reg [WIDTH-1:0] result,
    output zero
);

    always@* begin
        case (alu_op)
            3'b000: result = a - b; // ADD
            3'b001: result = a - b; // SUB
            3'b010: result = a & b; // AND
            3'b011: result = a | b; // OR
            3'b100: result = a ^ b; // XOR
            3'b101: result = a << b[$clog2(WIDTH)-1:0]; // SLL
            3'b110: result = a >> b[$clog2(WIDTH)-1:0]; // SRL
            3'b111: result = ($signed(a) < $signed(b)) ? 1 : 0; // SRA
            default: result = 0;
        endcase
    end

    assign zero = (result == 0);
endmodule
```

Figure 2 – Output code for Checkpoint 1

To fix the syntax error, we must declare result as a procedural variable (reg). This is because it is assigned in an **always @*** block, so any signal assigned in this block must be a procedural variable. Any output signal not declared as a procedural variable behave as a wire/net and must be assigned using **assign**.

Figure 1 above shows the terminal output after the syntax error has been fixed in the Verilog file. At this point, there is still a logical error, and several test cases are failing.

Checkpoint 2:

```
=== ALU VERIFICATION START ===
PASS @ line 1
PASS @ line 2
PASS @ line 3
PASS @ line 4
PASS @ line 5
PASS @ line 6
PASS @ line 7
PASS @ line 8
PASS @ line 9
PASS @ line 10
PASS @ line 11
PASS @ line 12
PASS @ line 13
PASS @ line 14
PASS @ line 15
PASS @ line 16
PASS @ line 17
PASS @ line 18
PASS @ line 19
PASS @ line 20
PASS @ line 21
PASS @ line 22
PASS @ line 23
PASS @ line 24
PASS @ line 25
PASS @ line 26
PASS @ line 27
PASS @ line 28
PASS @ line 29
PASS @ line 30
PASS @ line 31
PASS @ line 32
=== ALL TESTS PASSED (32 vectors) ===
```

Figure 3 – Terminal output showing passing testcases

```
`timescale 1ns/1ps

module alu #(
    parameter WIDTH = 32
)(
    input  [WIDTH-1:0] a,
    input  [WIDTH-1:0] b,
    input  [2:0]      alu_op,
    output reg [WIDTH-1:0] result,
    output           zero
);

always@* begin
    case (alu_op)
        3'b000: result = a + b;           // ADD
        3'b001: result = a - b;           // SUB
        3'b010: result = a & b;           // AND
        3'b011: result = a | b;           // OR
        3'b100: result = a ^ b;           // XOR
        3'b101: result = a << b[$clog2(WIDTH)-1:0]; // SLL
        3'b110: result = a >> b[$clog2(WIDTH)-1:0]; // SRL
        3'b111: result = ($signed(a) < $signed(b)) ? 1 : 0; // SRA
        default: result = 0;
    endcase
end

assign zero = (result == 0);

endmodule
```

Figure 4 – Verilog code for checkpoint 2

The logical error in the Verilog file occurs during the addition operation (code 3'b000). Instead of performing addition, the current file performs subtraction, which causes testcases to fail, as seen in Figure 1. When the file is updated to perform operation, all testcases pass as can be seen in Figure 2.

Checkpoint 3:

```
`timescale 1ns/1ps

module tb_alu_verify;

    localparam int WIDTH = 32;

    logic [WIDTH-1:0] a;
    logic [WIDTH-1:0] b;
    logic [2:0] alu_op;
    logic [WIDTH-1:0] result;
    logic zero;

    logic [WIDTH-1:0] exp_result;
    logic exp_zero;
    logic [3:0] op_field;

    integer fd;
    integer errors;
    integer line;

    // -----
    // DUT
    // -----
    alu #(
        .WIDTH(WIDTH)
    ) dut (
        .a(a),
        .b(b),
        .alu_op(alu_op),
        .result(result),
        .zero(zero)
    );

    // -----
    // Test procedure
    // -----
    initial begin

        fd = $fopen("alu_test_vectors.hex", "r");
        if (fd == 0)
            $fatal("ERROR: Could not open alu_test_vectors.hex");

        errors = 0;
        line = 0;

        $display("=== ALU VERIFICATION START ===");

        while (!$feof(fd)) begin
            line++;

            // Read fields EXACTLY as written
            if ($fscanf(fd,
                "%1h%8h%8h%8h%1h\n",
                op_field,
                a,
                b,
                exp_result,
                exp_zero
            ) != 5)
                continue;

            alu_op = op_field[2:0];

            #1; // combinational settle

            if (result != exp_result || zero != exp_zero) begin
                errors++;
                $display("FAIL @ line %0d", line);
                $display(" op=%b a=0x%08h b=0x%08h",
                    alu_op, a, b);
                $display(" expected: result=0x%08h zero=%b",
                    exp_result, exp_zero);
                $display(" got : result=0x%08h zero=%b",
                    result, zero);
            end
            else begin
                $display("PASS @ line %0d", line);
            end
        end

        $fclose(fd);

        if (errors == 0)
            $display("=== ALL TESTS PASSED (%0d vectors) ===", line);
        else
            $display("=== FAILED: %0d errors ===", errors);

        $finish;
    end

    //VCD Dump
    initial begin
        $dumpfile("alu_tb.vcd");
        $dumpvars(0, tb_alu_verify);
    end
endmodule
```

Figure 5 – Verilog testbench code for checkpoint 3

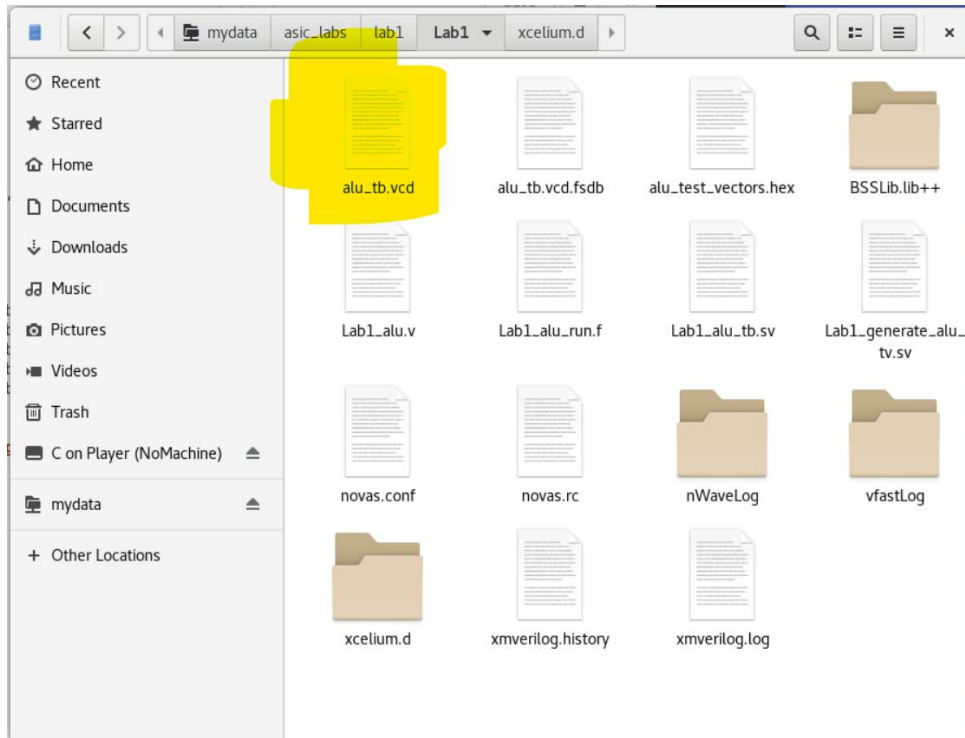


Figure 6 – alu_tb.vcd file created in working directory.

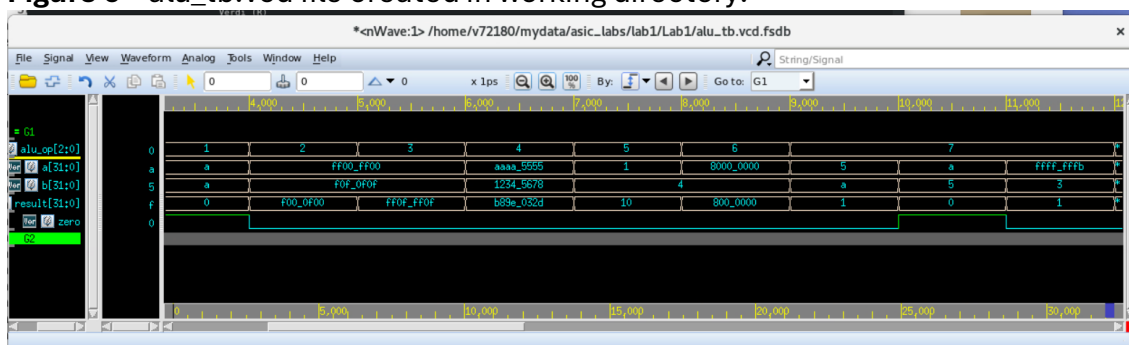


Figure 7 – Waveform contents seen in nWave

To create a .vcd file containing our results, we add a block to dump the signals in the file into **alu_tb.vcd**. After updating the testbench, the file can be seen in the working directory and inspected using nWave. The waveforms were manually inspected and shown to the TA for verification.